



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 微处理器原理与应用

实验名称: 实验 4 汇编程序提高实验

专 业 班 级 通信工程 三班

学 生 学 号

学 生 姓 名

实 验 时 间 2025 年 3 月 24 日

实验报告

一、 实验内容

1、编写一个汇编程序，实现字符串的复制功能, 并且将复制的字符串显示出来。（同时完成选做部分：倒序显示复制字符串）

我原本想要效仿上次实验中的高精度加法，利用 SI，DI 寄存器来直接指向字符串来进行复制的，但在查询相关资料之后，我找到了有关的字符串指令 LODSB 与 STOSB，它们能完美地完成基础要求，同时，配合上栈地先入后出的特性，倒叙显示字符串的功能也不难实现了。

画出流程图如下：

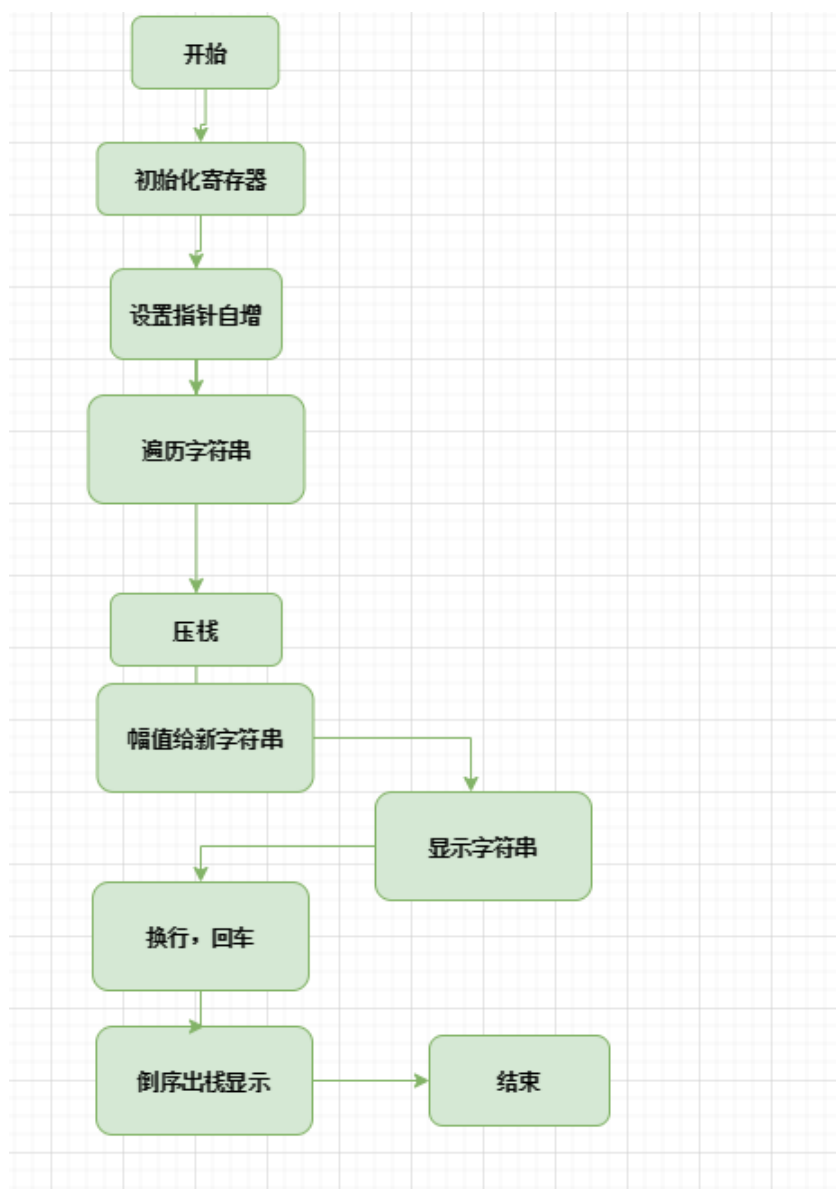


图 1

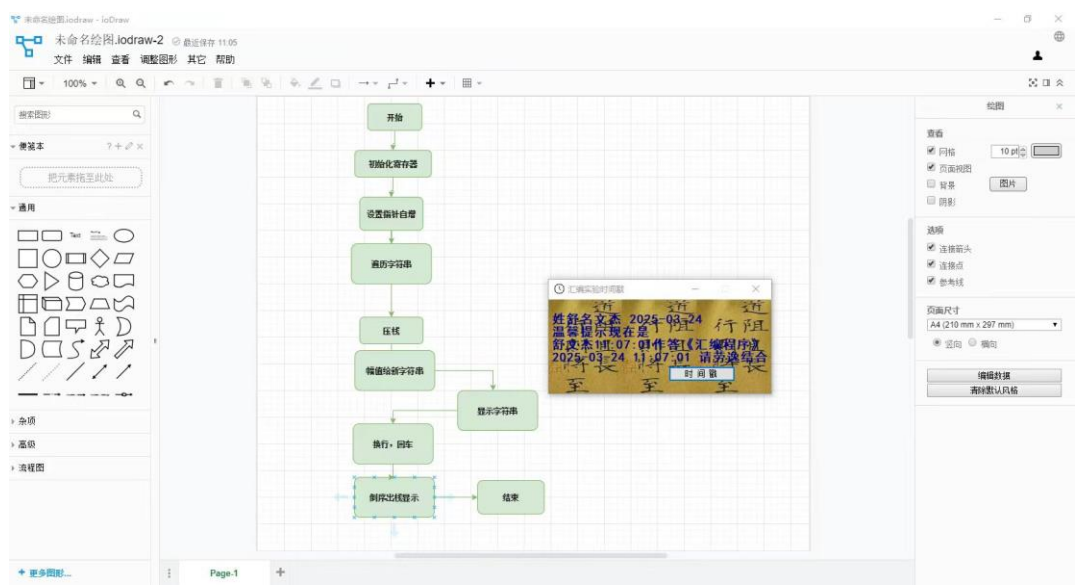


图 2

源代码如下：

1. DATAS SEGMENT ; 定义数据段
2. string_a DB 'the School of Information Science and Engineering, Shandong University','\$' ; 定义源字符串，以\$结尾
3. DATAS ENDS
- 4.
5. STACKS SEGMENT STACK ; 定义堆栈段
6. DB 200 DUP(?) ; 预留 200 字节堆栈空间
7. stack_top LABEL WORD ; 定义栈顶标签（地址标记）
8. STACKS ENDS
- 9.
10. EXT SEGMENT ; 定义附加段
11. string_b DB 100 DUP('\$') ; 定义目标字符串空间，预填充\$符号

12. EXT ENDS

13.

14. CODES SEGMENT ; 定义代码段

15. ASSUME CS:CODES,DS:DATAS,SS:STACKS,ES:EXT ; 指定段寄存器与段的关联关系

16. **START:** ; 程序入口点

17. MOV AX,DATAS ; 将 DATAS 段地址加载到 AX 寄存器

18. MOV DS,AX ; 将 AX 值赋给 DS，使 DS 指向 DATAS 段

19.

20. MOV AX,EXT ; 将 EXT 段地址加载到 AX 寄存器

21. MOV ES,AX ; 将 AX 值赋给 ES，使 ES 指向 EXT 段

22.

23. MOV AX, STACKS ; 将 STACKS 段地址加载到 AX 寄存器

24. MOV SS,AX ; 将 AX 值赋给 SS，设置堆栈段

25. LEA SP, stack_top ; 将栈顶地址加载到 SP 寄存器，初始化堆栈指针

26.

27. MOV SI,OFFSET string_a ; 将源字符串首地址存入 SI 寄存器

28. MOV DI,OFFSET string_b ; 将目标字符串首地址存入 DI 寄存器

29.

30. CLD ; 清除方向标志位，设置 SI/DI 自动递增

31.

32. XOR CX,CX ; 将 CX 寄存器清零，用于后续字符计数

33.

34.	COPY_LOOP:	; 字符串复制循环开始
35.	LODSB	; 从 SI 指向地址读取字符到 AL, SI 自动+1
36.	STOSB	; 将 AL 中的字符写入 DI 指向地址, DI 自动+1
37.	XOR AH,AH	; 清除 AH 寄存器 (配合 AX 压栈操作)
38.	PUSH AX	; 将 AX (当前字符) 压入堆栈保存
39.	INC CX	; CX 递增, 统计字符数量 (含结束符\$)
40.	CMP AL,'\$'	; 检查当前字符是否为结束符\$
41.	JNE COPY_LOOP	; 如果不是结束符, 继续循环
42.		
43.	MOV DX, OFFSET string_b	; 将目标字符串地址存入 DX 寄存器
44.	MOV AX, ES	; 将 ES (EXT 段) 值存入 AX 寄存器
45.	MOV DS, AX	; 将 AX 赋给 DS, 使 DS 指向 EXT 段 (准备输出)
46.	MOV AH,09H	; 设置 DOS 功能号 09H (显示字符串)
47.	INT 21H	; 调用 DOS 中断, 显示 string_b 内容
48.		
49.	MOV DL, 0DH	; 将回车符 ASCII 码存入 DL
50.	MOV AH, 02H	; 设置 DOS 功能号 02H (显示字符)
51.	INT 21H	; 调用 DOS 中断, 输出回车符
52.		
53.	MOV DL, 0AH	; 将换行符 ASCII 码存入 DL
54.	MOV AH, 02H	; 设置 DOS 功能号 02H (显示字符)
55.	INT 21H	; 调用 DOS 中断, 输出换行符

```

56.
57.  DEC CX
58.  POP DX                ; CX 减 1（排除结束符$的计数）
59.
60. IN_COPY_LOOP:        ; 逆序输出循环开始
61.  POP DX                ; 从堆栈弹出字符到 DX 寄存器（恢复字符顺序）
62.  MOV AH,02H           ; 设置 DOS 功能号 02H（显示字符）
63.  INT 21H              ; 调用 DOS 中断，输出 DL 中的字符
64.  LOOP IN_COPY_LOOP    ; CX 递减并循环，直到 CX=0
65.
66.  MOV AH,4CH           ; 设置 DOS 功能号 4CH（程序终止）
67.  INT 21H              ; 调用 DOS 中断，结束程序
68.
69. CODES ENDS
70.  END START            ; 程序结束，指定入口点为 START

```

二、 实验步骤



图表 1

1、用 windows 集成编译环境编写. ASM 源程序。

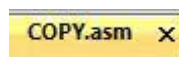


图表 2

2、运行测试代码。



直接无法查询到文件了，难道是名字太长了？



于是，我修改了它的名字：



图 3

重复运行代码：

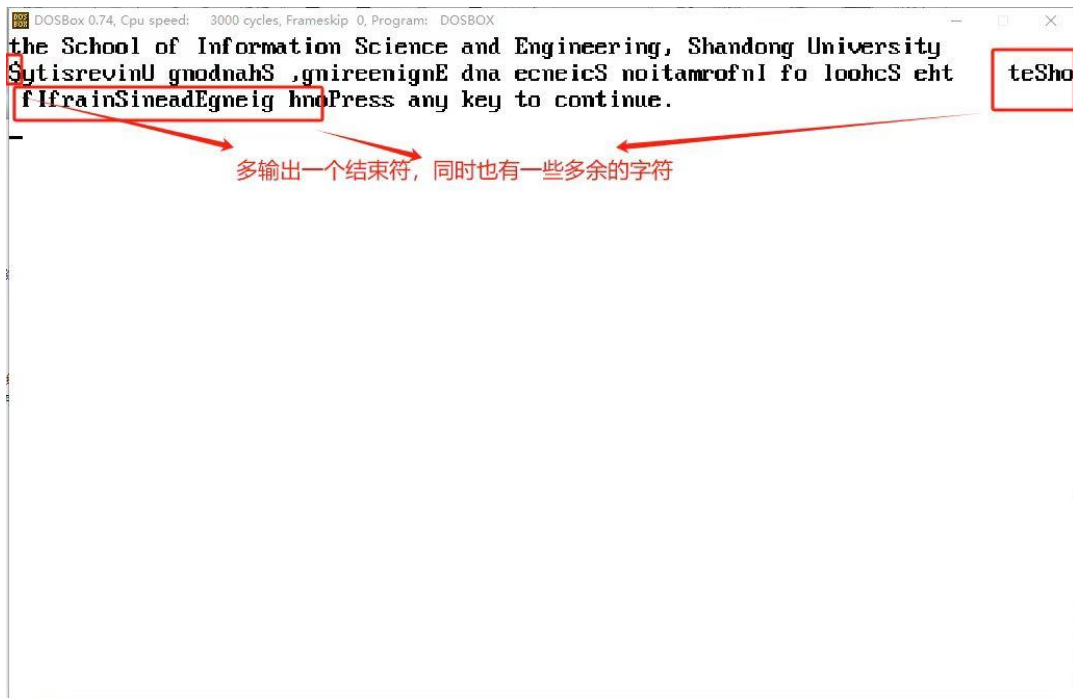


图 4

看来我们复制序列方面完成的不错, 但是倒叙显示有点问题。

开始调试:

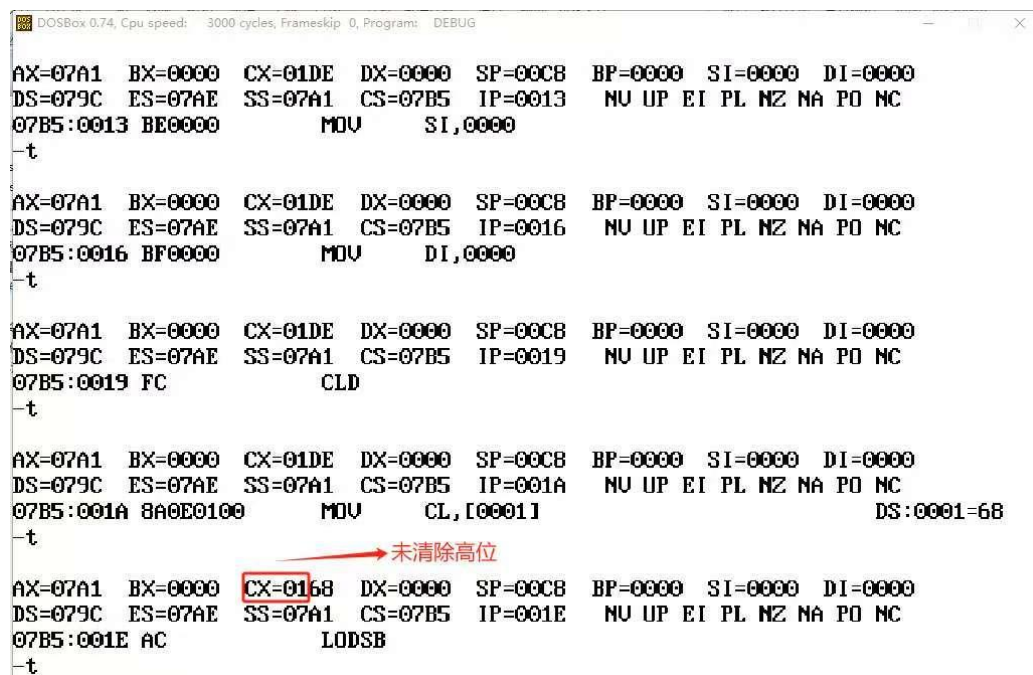


图 5

原来是未添加高位, 同时也复制了终止符。

```

30
31 CLD;
32 XOR CH,CH
33 MOV CL,[string_a+1]
34
35 COPY_LOOP:
36 LODSB
37 STOSB
38 XOR AH,AH
39 PUSH AX
40 INC CL → 直接循环自增
41 CMP AL,'?'
42 JNE COPY_LOOP
43
44 ; 显示复制后的字符串
45 MOV DX, OFFSET string_b ; DX = 目标字符串偏移地址
46 MOV AX, ES ; 将 ES (EXT段) 赋给 DS
47 MOV DS, AX
48 MOV AH,09H
49 MOV DX, OFFSET string_b
50 INT 21H
51
52 MOV DL, 0DH ; 回车符
53 MOV AH, 02H
54 INT 21H
55
56 MOV DL, 0AH ; 换行符
57 MOV AH, 02H
58 INT 21H
59
60 ;XOR CH,CH
61 ;DEC CX;
62 POP DX
63
64 IN COPY_LOOP:
65

```

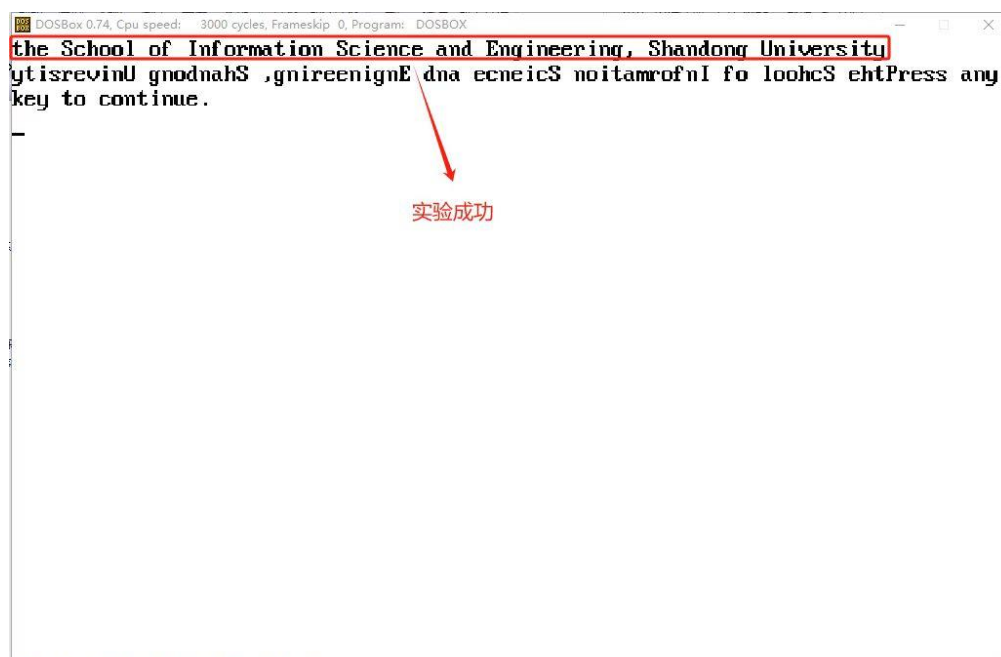
图 6

```

40 JNE COPY_LOOP
41
42 ; 显示复制后的字符串
43 MOV DX, OFFSET string_b ; DX = 目标字符串偏移地址
44 MOV AX, ES ; 将 ES (EXT段) 赋给 DS
45 MOV DS, AX
46 MOV AH,09H
47 MOV DX, OFFSET string_b
48 INT 21H
49
50 MOV DL, 0DH ; 回车符
51 MOV AH, 02H
52 INT 21H
53
54 MOV DL, 0AH ; 换行符
55 MOV AH, 02H
56 INT 21H
57
58 DEC CX; → 把终止符丢了
59 POP DX
60 IN_COPY_LOOP:
61 POP DX
62 MOV AH,02H
63 INT 21H
64 LOOP IN_COPY_LOOP
65
66 ; 退出程序
67 MOV AH, 4CH
68
69

```

图 7



实验成功

2、利用中断调用，在屏幕上显示1—9 之间随机数。中断号86H

根据老师所讲的定义法，我们可以先计算相应的中断向量表86H所处的位置，再将中断子程序地址写进去，同时，我查询过资料，86H地址并没有对应的指令，于是我便没有做子程序的保护。

对于DOS法设置中断子程序，我们将子程序的偏移地址送进DX，要设置的中断号送入AL，最后调用25H然后INT21H便可以设置成功。同时，我查询过资料，86H地址并没有对应的指令，于是我便没有用35H做子程序的保护。

①定义法：

画出流程图：

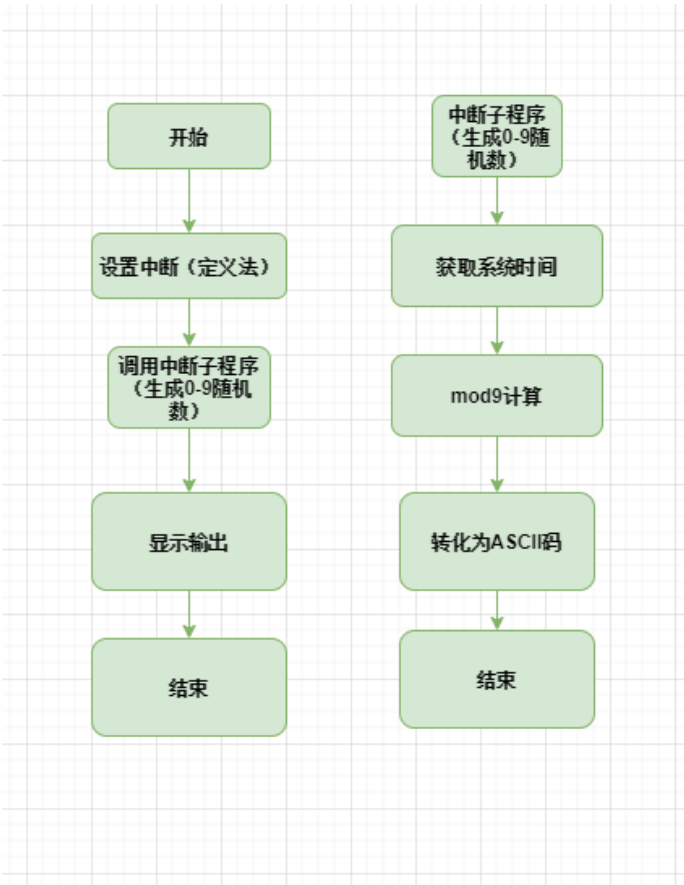


图 8

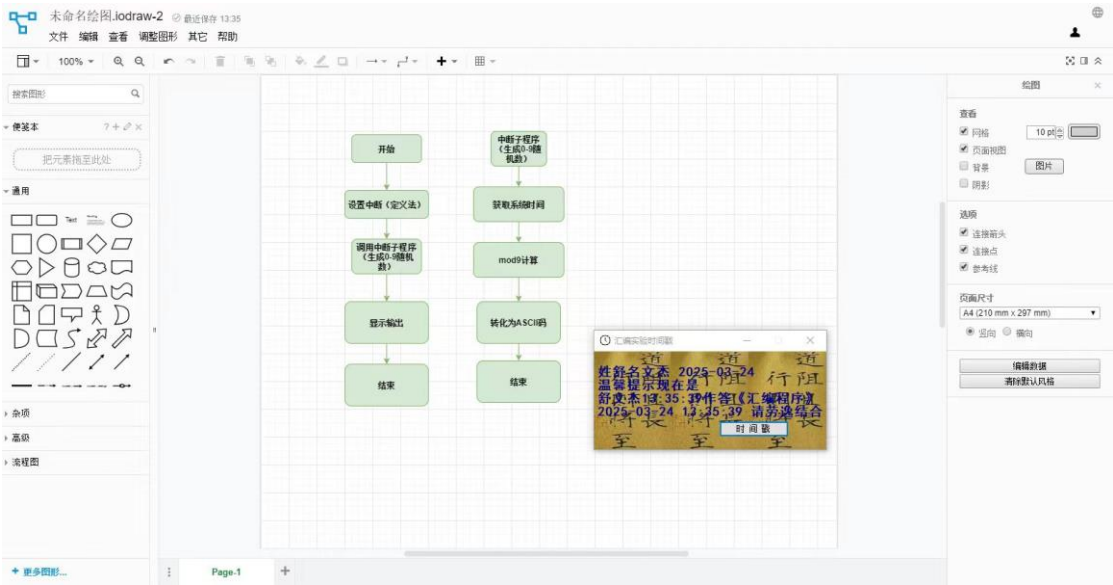


图 9

源代码如下：

```

1. CODES SEGMENT
2.     ASSUME CS:CODES    ; 声明代码段寄存器关联
3.     ORG 100H           ; 程序起始地址设为 100H (COM 文件格式)
4.     START:
5.     MOV AX,CS          ; 将代码段地址存入 AX
6.     MOV DS,AX          ; 设置 DS = CS (COM 文件数据段与代码段重叠)
7.
8.     ; 定义法设置中断向量表, 绑定 86H 号中断到自定义处理程序
9.     XOR AX,AX          ; AX 清零
10.    MOV ES,AX           ; ES 指向中断向量表所在段 (0000H)
11.    MOV BX,86H*4        ; 计算中断号 86H 的向量地址 (每个中断占 4 字节)
12.    MOV AX,OFFSET RANDOM_INT ; 获取中断处理程序偏移地址
13.    MOV WORD PTR ES:[BX],AX ; 写入偏移地址到向量表
14.    MOV AX,SEG RANDOM_INT ; 获取中断处理程序段地址
15.    MOV WORD PTR ES:[BX+2],AX ; 写入段地址到向量表
16.
17.    INT 86H             ; 手动触发 86H 号中断 (调用 RANDOM_INT)
18.    MOV AH,02H          ; 设置 DOS 功能号 02H (显示字符)
19.    INT 21H             ; 调用 DOS 中断 (此时 DL=随机数 ASCII 码)
20.

```

```

21.  MOV AH,4CH      ; 设置 DOS 功能号 4CH (程序终止)

22.  INT 21H        ; 退出程序

23.

24. ; 自定义中断处理程序: 生成 1-9 的随机数 ASCII 码

25. RANDOM_INT PROC

26.  PUSH AX        ; 保存寄存器 (未完全保存, 存在潜在问题)

27.  MOV AH,00H     ; BIOS 中断 1AH 功能 00H (读取系统计时器)

28.  INT 1AH        ; 返回 DX=计时器低字, CX=高字

29.  MOV AX,DX      ; 将随机数种子存入 AX

30.  MOV BX,9       ; 除数 BX=9

31.  XOR DX,DX      ; 清零 DX (准备进行 32 位除法)

32.  DIV BX         ; AX/BX, 余数存入 DX (范围 0-8)

33.  INC DX         ; 余数+1 (范围 1-9)

34.  ADD DL,'0'     ; 转换为 ASCII 字符 ('1'-'9')

35.  POP AX         ; 恢复寄存器

36.  IRET           ; 中断返回

37. RANDOM_INT ENDP

38.

39. CODES ENDS

40.  END START      ; 程序结束, 入口点为 START

```

1. 在基础编译环境中编写汇编代码。

```

01 ;默认采用ML6.11汇编程序
02 DATAS SEGMENT
03 ;此处输入数据段代码
04 DATAS ENDS
05
06 STACKS SEGMENT
07 ;此处输入堆栈段代码
08 STACKS ENDS
09
10 CODES SEGMENT
11 ASSUME CS:CODES,DS:DATAS,SS:STACKS
12 START:
13 MOV AX,DATAS
14 MOV DS,AX
15 ;此处输入代码段代码
16 MOV AH,4CH
17 INT 21H
18 CODES ENDS
19 END START
    
```

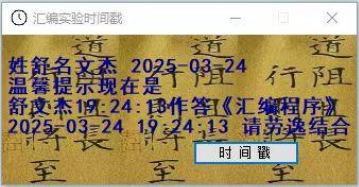


图 10

```

01 CODES SEGMENT
02 ASSUME CS:CODES
03 ORG 100H
04 START:
05 MOV AX,CS
06 MOV DS,AX
07
08
09 ; 设置中断向量
10 XOR AX, AX
11 MOV ES, AX ; ES指向中断向量表
12 MOV BX, 86H*4 ; 中断号86H的向量地址
13 MOV AX, OFFSET RANDOM_INT
14 MOV WORD PTR ES:[BX], AX ; 设置偏移
15 MOV AX, SEG RANDOM_INT
16 MOV WORD PTR ES:[BX+2], AX ; 设置段
17
18 INT 86H
19 MOV AH,02H
20 INT 21H
21
22 MOV AH,4CH
23 INT 21H
24
25 RANDOM_INT PROC
26 MOV AH, 00H
27 INT 1AH
28 MOV AX, DX
29 MOV BX, 9
30 XOR DX, DX
    
```

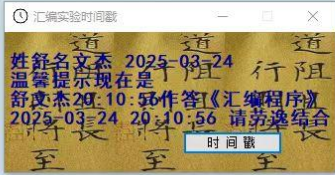


图 11

2. 运行代码



图 12

实验成功。

②DOS法

画出流程图：

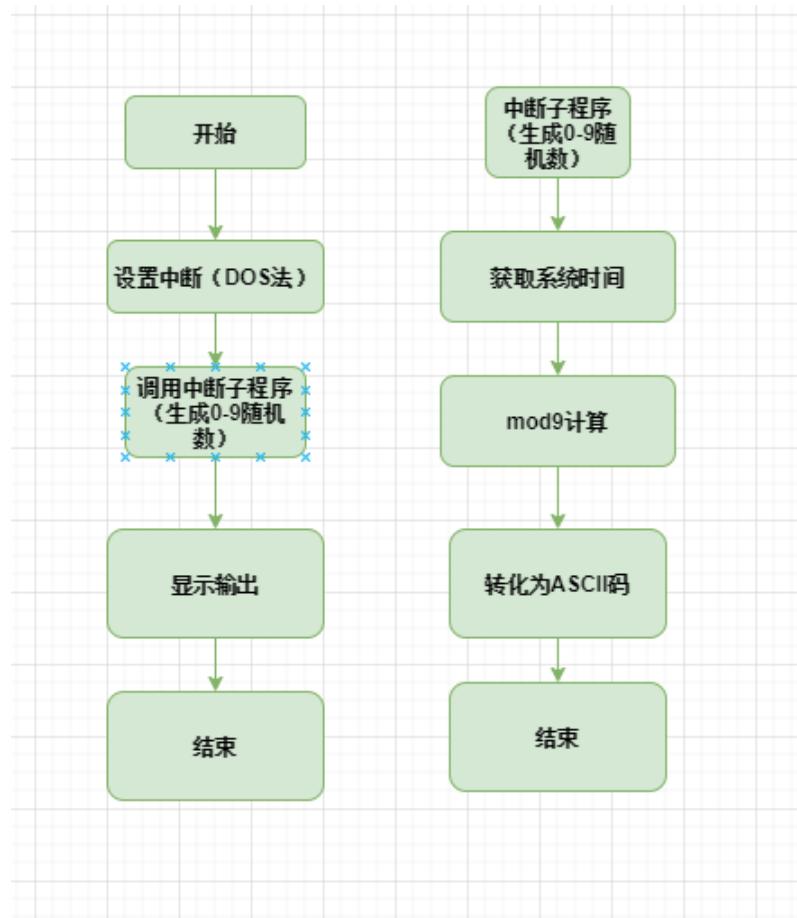


图 13

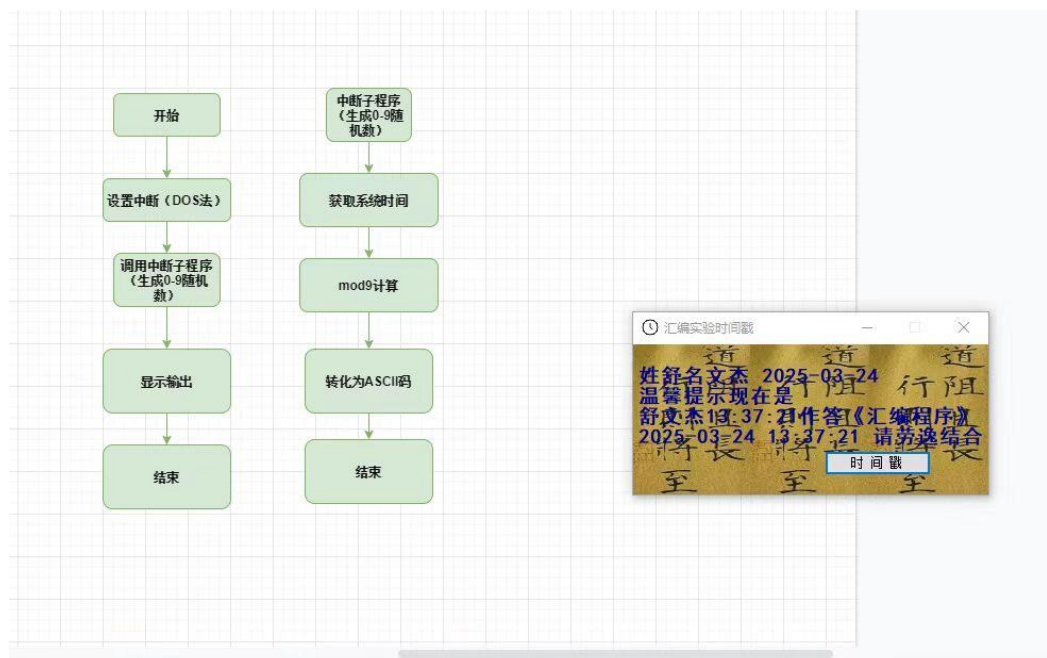


图 14

源代码如下：

```

1. CODES SEGMENT
2.     ASSUME CS:CODES
3.     ORG 100H           ; COM 程序格式，起始地址 100H
4.     START:
5.     MOV AX,CS
6.     MOV DS,AX         ; 设置 DS=CS (COM 程序数据段与代码段重叠)
7.
8.     ; 使用 DOS 功能 25H 设置中断向量 (更安全的方式)
9.     MOV DX,OFFSET RANDOM_INT ; 获取中断处理程序偏移地址
10.    MOV AL,86H         ; 要设置的中断号
11.    MOV AH,25H         ; DOS 功能号 25H (设置中断向量)
12.    INT 21H           ; 调用 DOS 服务
13.
14.    INT 86H           ; 手动触发 86H 号中断
15.    MOV AH,02H         ; DOS 功能号 02H (显示字符)
16.    INT 21H           ; 显示 DL 中的 ASCII 字符
17.
18.    MOV AH,4CH         ; DOS 功能号 4CH (程序终止)
19.    INT 21H           ; 退出程序
    
```

```

20.

21. ; 自定义中断处理程序（需声明为 FAR 过程）

22. RANDOM_INT PROC FAR

23.     PUSHA                ; 保存所有通用寄存器（8086 不支持，需手动保存）

24.     MOV AH,00H           ; BIOS 功能 00H（读取系统计时器）

25.     INT 1AH              ; 返回 CX:DX=计时器值

26.     MOV AX,DX            ; 使用低字作为随机数种子

27.     MOV BX,9             ; 除数 BX=9

28.     XOR DX,DX            ; 清零 DX（准备除法）

29.     DIV BX               ; AX/BX，余数在 DX（0-8）

30.     INC DX               ; 余数+1（1-9）

31.     ADD DL,'0'           ; 转换为 ASCII 字符

32.     POPA                 ; 恢复所有寄存器（需与 PUSH A 匹配）

33.     IRET                 ; 中断返回

34. RANDOM_INT ENDP

35.

36. CODES ENDS

37.     END START
    
```

1. 在集成编译环境中编写汇编代码

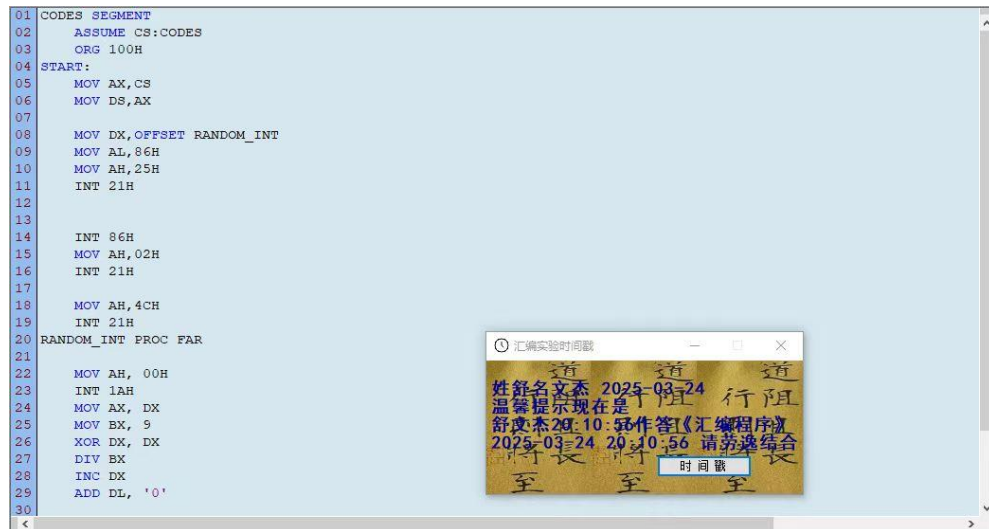


图 15

2. 运行相关代码



图 16

实验成功

3、键盘输入10 个学生的成绩，编写一个程序统计60-69 分， 70-79 分，80-89 分，90-99 分及100 分的人数，分别存放在Score6, Score7, Score8, Score9 和Score10 单元中。

【输入10 个学生的成绩如下】 65 98 78 82 88 95 72 62 90 100

对于这个程序的编写，很明显的我们得用多层分支来实现，但难点并不在此，问题是如何在这么长的字符串里提取字符长度不定的数字。我的想法是以检测ASCLL码的大小，判断字符的种类，从第一个数字字符开始，终止于非数字字符，便可以分割出数字，在将其转化为数字，再与设定值比较，相应的变量自增。在检测到回车后，便输出结果。

画出流程图如下：

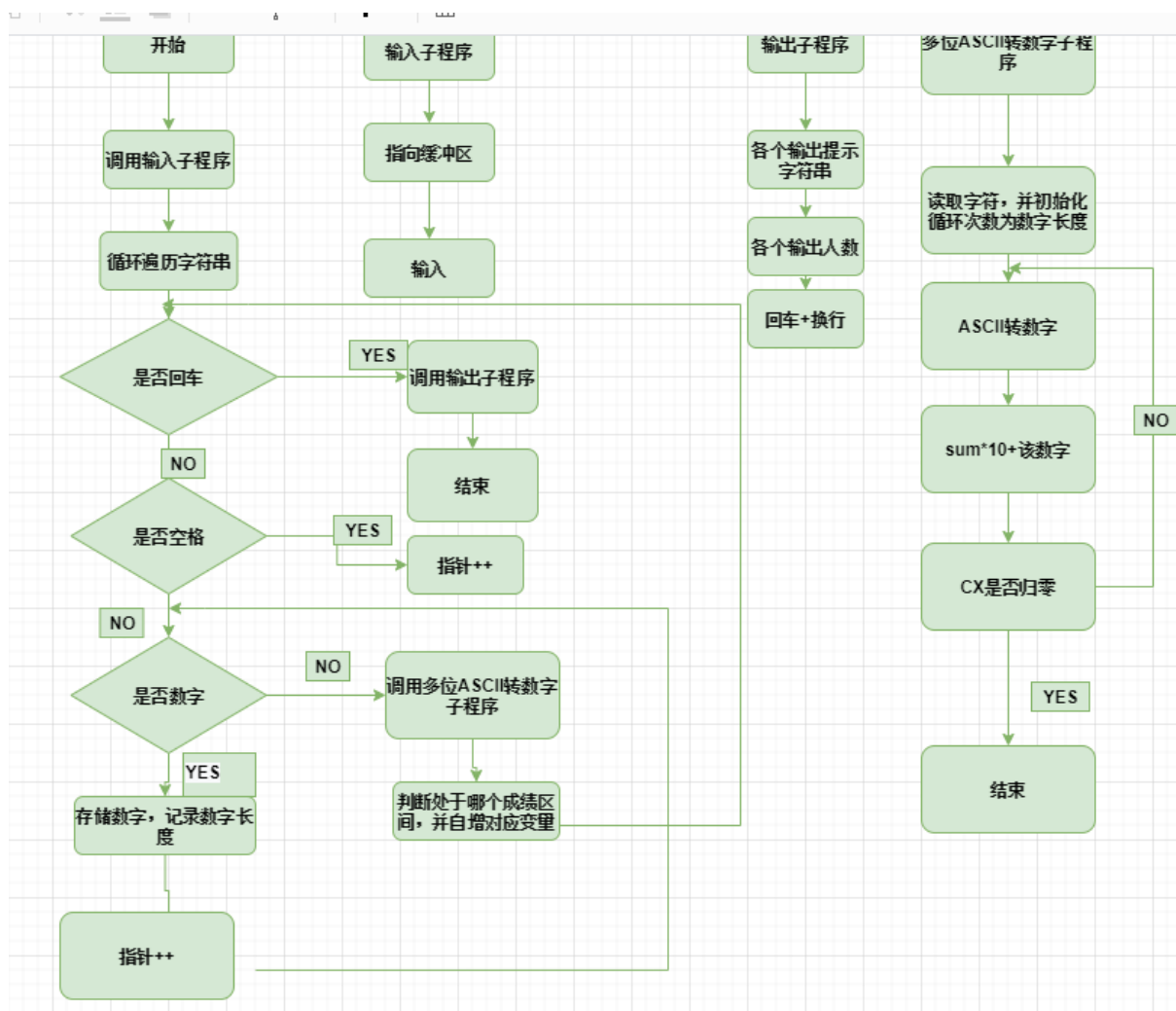


图 17



1. DATAS SEGMENT
2. ; 分数段计数器初始化正确
3. SCORE6 DB 0 ; 60-69 分计数
4. SCORE7 DB 0 ; 70-79 分计数
5. SCORE8 DB 0 ; 80-89 分计数
6. SCORE9 DB 0 ; 90-99 分计数
7. SCORE10 DB 0 ; 100 分计数
- 8.
9. ; 输入缓冲区结构符合 DOS 0AH 功能要求
10. BUF DB 40 ; 最大长度
11. DB ? ; 实际输入长度
12. DB 40 DUP(0) ; 输入存储区
- 13.
14. ; 临时变量定义合理

```

15.  num_length DB 1 ; 数字位数存储
16.  temp_num DB 5 DUP(0) ; 数字转换缓冲区
17.
18.  ; 输出信息定义正确
19.  msg60_69 DB '60-69: $'
20.  msg70_79 DB '70-79: $'
21.  msg80_89 DB '80-89: $'
22.  msg90_99 DB '90-99: $'
23.  msg100 DB '100: $'
24. DATAS ENDS
25.
26. STACKS SEGMENT
27.  DB 200 DUP(?)
28. STACKS ENDS
29.
30. CODES SEGMENT
31.  ASSUME CS:CODES,DS:DATAS,SS:STACKS
32.
33. START:
34.  MOV AX,DATAS
35.  MOV DS,AX ; 正确初始化数据段
36.

```



```

37.  CALL INPUTS    ; 调用输入子程序

38.  LEA SI,[BUF+2] ; SI 指向输入字符串起始位置

39.  XOR CH,CH

40.  MOV CL,11      ; 设置循环次数为 11（假设输入长度≤11）

41.

42.  PROCESS_LOOP:

43.  PUSH CX        ; 保存外层循环计数器

44.  CHECK_SP:

45.  MOV AL,[SI]

46.  CMP AL,' '     ; 检测空格

47.  JNE CHECK_ENTER

48.  INC SI         ; 跳过空格

49.  JMP CHECK_SP   ; 继续检查下一个字符

50.

51.  CHECK_ENTER:

52.  MOV AL,[SI]

53.  CMP AL,0DH     ; 检测回车符

54.  JNE CHECK_DIG

55.  CALL hh        ; 换行

56.  CALL PRINT_SC  ; 输出统计结果

57.  MOV AH,4CH

58.  INT 21H        ; 程序退出
    
```

59.

60. CHECK_DIG:

61. MOV DI,OFFSET temp_num

62. XOR BX,BX ; 数字字符计数器清零

63. CO_LOOP:

64. MOV AL, [SI]

65. CMP AL, '0' ; 数字有效性检查

66. JB END_IN ; <'0'则结束

67. CMP AL, '9'

68. JA END_IN ; >'9'则结束

69. MOV [DI+BX],AL ; 存储数字字符

70. INC BL ; 数字位数+1

71. INC SI ; 指针后移

72. JMP CO_LOOP

73.

74. END_IN:

75. MOV num_length, BL ; 保存数字长度

76. CALL TRANS ; 转换为二进制数 AX

77.

78. ; 分数段判断逻辑

79. CMP AX,69

80. JBE score6_case ; ≤69

```

81.  CMP AX,79
82.  JBE score7_case ; 70-79
83.  CMP AX,89
84.  JBE score8_case ; 80-89
85.  CMP AX,99
86.  JBE score9_case ; 90-99
87.  CMP AX,100
88.  JE score10_case ; 仅 100 分
89.  ; 此处未处理>100 的情况，会漏判后续改进
90.
91. score6_case:
92.  INC SCORE6
93.  POP CX
94.  LOOP PROCESS_LOOP ; 继续处理
95.
96. score7_case:
97.  INC SCORE7
98.  POP CX
99.  LOOP PROCESS_LOOP
100.
101. score8_case:
102.  INC SCORE8

```

```

103. POP CX
104. LOOP PROCESS_LOOP
105.
106. score9_case:
107. INC SCORE9
108. POP CX
109. LOOP PROCESS_LOOP
110.
111. score10_case:
112. INC SCORE10
113. POP CX
114. LOOP PROCESS_LOOP
115.
116. INPUTS PROC
117. LEA DX,BUF
118. MOV AH,0AH ; DOS 缓冲输入功能
119. INT 21H
120. RET
121. INPUTS ENDP
122.
123. TRANS PROC ; 字符串转数字
124. PUSH BX
    
```

```

125.  PUSH CX
126.  PUSH SI
127.  MOV SI, OFFSET temp_num
128.  XOR AX,AX    ; 结果寄存器清零
129.  MOV CL, [num_length]
130.  convert_loop:
131.  MOV BL, [SI]  ; 取字符
132.  SUB BL,'0'   ; 转数字
133.  MOV BH,0
134.  MOV DX,AX    ; 保存当前结果
135.  MOV AX,10    ; 乘数
136.  MUL DX      ; AX = AX*10
137.  ADD AX,BX    ; 加新数字
138.  INC SI
139.  LOOP convert_loop
140.  POP SI
141.  POP CX
142.  POP BX
143.  RET
144. TRANS ENDP
145.
146. PRINT_SC PROC    ; 输出结果（支持 0-9）
    
```

```

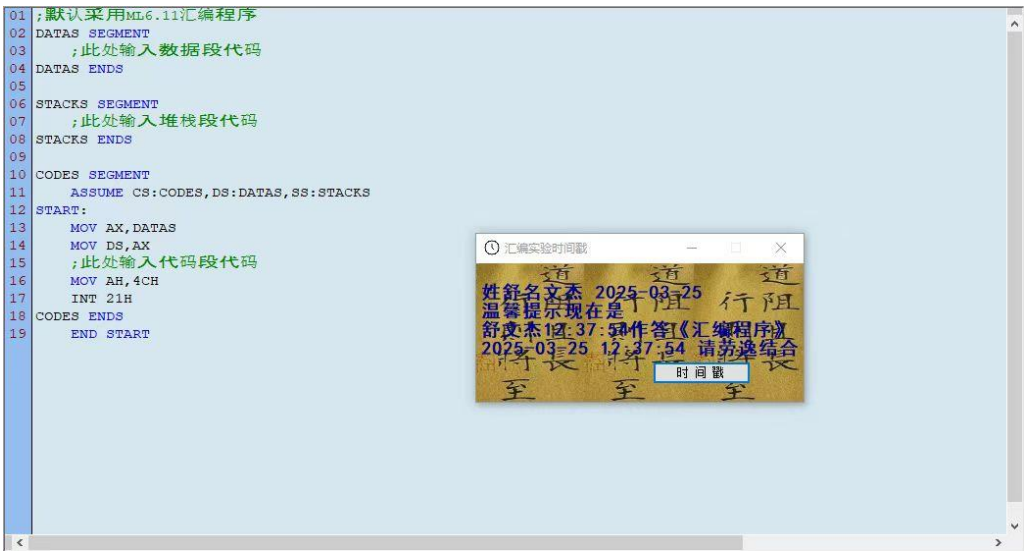
147.  MOV AH,09H
148.  MOV DX,OFFSET msg60_69
149.  INT 21H
150.  MOV DL,[SCORE6]
151.  ADD DL,'0'    ; 转换为ASCII
152.  MOV AH,02H
153.  INT 21H
154.  CALL hh      ; 换行
155.
156.  ; 重复类似结构输出其他分数段...
157.  RET
158. PRINT_SC ENDP
159.
160. hh PROC      ; 换行子程序
161.  MOV DL,0DH
162.  MOV AH,02H
163.  INT 21H
164.  MOV DL,0AH
165.  INT 21H
166.  RET
167. hh ENDP
168.

```

169. CODES ENDS

170. **END** START

1. 在集成编译环境里面编写程序：



The screenshot shows an IDE with assembly code on the left and a terminal window on the right. The code is as follows:

```
01 ;默认采用ML6.11汇编程序
02 DATAS SEGMENT
03 ;此处输入数据段代码
04 DATAS ENDS
05
06 STACKS SEGMENT
07 ;此处输入堆栈段代码
08 STACKS ENDS
09
10 CODES SEGMENT
11 ASSUME CS:CODES,DS:DATAS,SS:STACKS
12 START:
13     MOV AX,DATAS
14     MOV DS,AX
15     ;此处输入代码段代码
16     MOV AH,4CH
17     INT 21H
18 CODES ENDS
19     END START
```

The terminal window displays the output of the program, which includes the name '文杰', the date '2025-03-25', and the time '12:37:54'. It also shows a prompt '请输入' (Please enter) and a button labeled '时间戳' (Timestamp).

图 19



The screenshot shows an IDE with assembly code on the left and a terminal window on the right. The code is as follows:

```
001 DATAS SEGMENT
002     SCORE6 DB 0
003     SCORE7 DB 0
004     SCORE8 DB 0
005     SCORE9 DB 0
006     SCORE10 DB 0
007     BUF DB 40 ; 输入缓冲区
008     DB ?
009     DB 40 dup(0)
010     num_length DB 1
011     temp_num db 5 dup(0)
012     msg60_69 db '60-69: $'
013     msg70_79 db '70-79: $'
014     msg80_89 db '80-89: $'
015     msg90_99 db '90-99: $'
016     msg100 db '100: $'
017 DATAS ENDS
018
019 STACKS SEGMENT
020     DB 200 DUP(?)
021 STACKS ENDS
022
023 CODES SEGMENT
024 ASSUME CS:CODES,DS:DATAS,SS:STACKS
025 START:
026     MOV AX,DATAS
027     MOV DS,AX
028
029     CALL INPUTS
030     LEA SI,[BUF+2]
```

The terminal window displays the output of the program, which includes the name '文杰', the date '2025-03-25', and the time '17:25:49'. It also shows a prompt '请输入' (Please enter) and a button labeled '时间戳' (Timestamp).

图 20

2. 运行代码



实验成功。

但是在这个程序中，我默认了输入的数据都是在60-100的，并没有对低于60与高于100的数据进行越界判断，后续可以再加上两个判断进行改进。

【实验心得】

时间飞逝，转眼已是最后一次汇编实验。相较于之前的实验，这次的任务让我感受到明显的从容——或许是前五次实验的积累让我的汇编基础扎实了许多，又或许是解决问题的思维方式逐渐向底层逻辑靠拢。这次实验中的每一个环节，都让我更深刻地体会到：汇编语言的简洁与复杂往往并存，而唯有对细节的掌控才能让代码真正“听话”。

从“笨拙”到“精巧”：指令的妙用

在字符串复制与倒序输出中，我最初尝试用`MOVSB`指令逐字节搬运数据，但发现无法兼顾复制和倒序的需求——若直接覆盖原字符串，倒序时会丢失数据。突然灵光一现，想到`LODSB`（从`SI`加载字符到`AL`）和`STOSB`（将`AL`存入`DI`）这对指令：它们不仅能自动递增指针，还能通过`AL`寄存器暂存数据，为后续压栈倒序提供了可能。这一优化让代码从原本冗长的循环中解脱出来，仅用十余行便实现了功能。然而，调试时发现倒序输出的首字符总多出一个乱码，通过`DEBUG`单步跟踪发现，结束符`\$`也被压入栈中，导致弹出时被误显示。于是，我在倒序循环前加入`DEC CX`（排除`\$`计数），问题迎刃而解。这一教训让我明白：汇编中的“边界”从不是模糊的概念，而是精确到每个字节的算计。

中断：在“规则”与“自由”间博弈

随机数生成实验让我对中断机制有了更深的理解。起初，我试图用 INT 1AH 读取时钟计数器直接作为随机数，但发现数值范围远超预期（0~65535）。于是，我引入除法取余算法：将计数器低字除以 9，余数范围 0~8，再加 1 映射到 1~9。但测试时发现某些情况下结果始终为 0——原来当计数器低字为 0 时，除法余数也为 0，导致结果错误。最终通过将`DX`（余数寄存器）初始化为随机种子，而非直接使用`AX`，解决了这一问题。此外，在设置中断向量时，我原本直接修改内存中的向量表（定义法），但意识到可能存在与其他程序的冲突风险后，改用 DOS 的 25H 功能（设置向量）和 35H 功能（保存原向量），兼顾了安全性与兼容性。这让我意识到：中断是硬件留给软件的“后门”，但只有遵循规则，才能让自由不越界。

成绩统计：输入处理的“分形”挑战

多分数段统计实验看似简单，实则暗藏玄机。输入字符串如 65 98 78 82... 需要被解析为多个独立数值，而汇编中缺乏现成的字符串分割函数。我的解决思路是：用`SI`遍历输入缓冲区，遇到数字字符时启动内层循环，将连续数字存入临时数组，再调用`TRANS`子程序（通过 $AX = AX * 10 + (\text{字符} - '0')$ ）转换为二进制数。此外，输入中的空格和回车需精确跳过，我设计了一个 CHECK_SP 子循环，通过 JNE 和 INC SI 的组合实现“智能”指针推进。这一过程让我体会到：汇编中的输入处理如同破解密码，每一步都需要预设所有可能的“陷阱”。

从“能跑”到“优雅”：代码的进化

回顾整个实验，我逐渐从“功能实现”转向“代码优化”。例如，在成绩统计中，最初我将所有分支判断（如`CMP AX,69`、`CMP AX,79`）线性排列，导致代码臃肿。后来改用“边界链式比较”（如先用`CMP AX,89`判断是否进入 80-89 段，否则进入更低分段），减少了跳转次数。此外，我将换行功能封装为`hh`子程序，输入解析和数值转换分离为 INPUTS 和 TRANS 模块，提升了代码复用率。这种“模块化”思维不仅让调试更高效，也为未来扩展（如支持三位数成绩）预留了空间。

结语：汇编的“哲学课”

这六次实验犹如一场漫长的修行，让我从“敬畏”汇编的晦涩，到“享受”其直击硬件的纯粹。如果说高级语言是驾驭汽车的便利，汇编则是亲手打造每一个零件的匠人体验——它教会我：真正的控制源于对细节的极致掌握，而每一行代码都是对机器逻辑的虔诚翻译。未来若再遇底层性能优化或嵌入式开发，这段“痛苦”而充实的汇编之旅，必将成为我技术生涯中最坚实的基石。